Ride-Hailing Surge Pricing Engine: A Concurrent Dynamic-Pricing System Using Greedy and Dynamic Programming Algorithms with AsyncIO

Vundru Venkata Sai Durgesh
Scholar, M.Tech, Department of Computer Science and Engineering
Koneru Lakshmaiah Education Foundation
Vaddeswaram, Guntur, Andhra Pradesh, India
2601050109@kluniversity.in

Raju Anitha
Professor, Department of Computer Science and Engineering
Koneru Lakshmaiah Education Foundation
Vaddeswaram, Guntur, Andhra Pradesh, India
anitharaju@kluniversity.in

***Abstract*—Ride-hailing platforms use surge pricing to balance ride requests against available drivers, but mapping the instantaneous demand/supply ratio directly to a fare multiplier makes prices unstable and prone to sudden spikes. This paper presents a concurrent surge-pricing engine written in Python 3.12 that combines a greedy tier rule, dynamic programming and AsyncIO. For each of five simulated city zones, a greedy algorithm selects one of five multipliers (1.0× to 2.5×) from the demand/supply ratio. A dynamic-programming layer memoises solved (zone, demand, supply) states and applies a per-zone smoothing recurrence that bounds how fast the multiplier can change. Zones are processed simultaneously by producer tasks, an asynchronous queue and a pool of worker tasks, and the results are exposed through a thread-safe price board to a Streamlit dashboard. In a seeded simulation of 300 ticks per zone, smoothing reduced the mean absolute tick-to-tick change of the multiplier by about 22% (0.0505 to 0.0395), eliminated all 59 jumps of 0.5× or more and left the mean multiplier essentially unchanged. Processing five zones concurrently took 0.53 s against 2.5 s sequentially, and an automated suite of 68 test cases passed. The results show that a transparent, rule-based pricing policy can be made stable and scalable without opaque models.**

***Index Terms*—surge pricing, dynamic pricing, greedy algorithm, dynamic programming, memoisation, AsyncIO, ride-hailing.**

I. Introduction

Ride-hailing platforms such as Uber, Ola and Rapido connect riders with nearby drivers through a mobile application. A basic difficulty of such two-sided marketplaces is that demand and supply are rarely balanced: during office closing hours, heavy rain or large events the number of requests in a locality can grow several times within minutes, while the number of free drivers does not. Surge pricing restores balance by multiplying the normal fare by a factor greater than one when demand exceeds supply, so that less urgent riders postpone their trip and drivers are attracted to the busy zone.

The importance of this mechanism is supported by recent evidence. Liu *et al.* estimated customer and driver price elasticities from a large data set of a Chinese ride-hailing platform and concluded that platforms can improve profits by confining surge pricing to peak hours and by raising driver compensation [1]. Castillo *et al.* showed that high demand can trigger a harmful feedback cycle, the "wild goose chase", in which few idle drivers, long pick-up times and low earnings reduce welfare, and that raising prices at high-demand times brings demand back under control and performs better than adjustments to the matching algorithm [2], [3].

Designing the software that produces the price signal is itself an engineering problem. A direct mapping from the instantaneous demand/supply ratio to a multiplier has four weaknesses: (i) it is unstable, because a small random change near a tier boundary changes the price; (ii) it produces sudden spikes when a short burst of requests pushes the multiplier to its maximum in one step; (iii) it repeats work, because identical situations are evaluated again and again; and (iv) when zones are processed one after another, the price of the last zone is always older than that of the first.

This paper presents the Ride-Hailing Surge Pricing Engine, a compact and fully executable system that addresses these weaknesses. The contributions are: (1) a greedy tier policy, stored as data, that converts the demand/supply ratio into one of five multipliers; (2) a dynamic-programming layer consisting of a bounded memo table of (zone, demand, supply) states and a per-zone smoothing recurrence; (3) a concurrent architecture with one AsyncIO producer task per zone, a bounded queue and a worker pool; and (4) an object-oriented implementation with a Streamlit dashboard and an automated test suite, together with a quantitative evaluation of price stability, cache behaviour and concurrency.

The rest of the paper is organised as follows. Section II reviews related work, Section III describes the architecture, Section IV explains the algorithms, Section V describes the experimental setup, Section VI reports the results, Section VII discusses limitations, and Section VIII concludes.

II. Related Work

A. Economic and Operational Studies

Hall *et al.* analysed Uber data and found that surge pricing increases the supply of drivers in a surging area and reduces the number of unfulfilled requests [4]. Chen and Sheldon showed that drivers respond to higher earnings by working more hours, so the multiplier influences supply as well as demand [5]. Cohen *et al.* used the discontinuities created by discrete multipliers to estimate consumer surplus and concluded that dynamic pricing creates substantial value for riders who need a trip most [6]. Banerjee *et al.* proposed a queueing-theoretic model of ride-share platforms and showed that dynamic prices can achieve most of the welfare of an ideal state-dependent policy while being robust to errors in the estimation of demand and supply [7]. This result motivates the simple ratio-based policy used here: an explainable rule is acceptable if it is combined with sensible limits and smoothing.

Yan *et al.* studied dynamic pricing jointly with matching in ride-hailing platforms [8], and Freund and van Ryzin examined the limitations of dynamic pricing mechanisms in

ride-hailing [9]. Together with [1]–[3], these works establish why a platform needs a reliable pricing signal, but they are analytical or empirical and do not supply a reference implementation.

B. Algorithmic Pricing Approaches

Yuan and Van Hentenryck proposed a spatio-temporal pricing framework based on model-predictive control that works at a coarser temporal and spatial granularity than real-time dispatching and regulates demand so that riders are served within a reasonable time; simulations indicated short waiting times without sacrificing revenue [10]. Huang *et al.* modelled dynamic subsidies for non-stationary stochastic demand as a Markov decision process, combined change-point detection with a deterministic-policy deep reinforcement learning algorithm, and reported that the profitability of the subsidy strategy increases with the non-stationary level [11]. Gujral *et al.* introduced FairRide, a cooperative-game framework for fair surge pricing, and a variant that tempers volatility through temporal smoothing; on a synthetic data set of ten zones it reduced the incidence of extreme surges to below 8% [12].

These approaches rely on forecasting, optimisation or learning and need substantial data, training or computation. The present work takes the complementary route of keeping a transparent tier rule and obtaining stability from a lightweight dynamic-programming recurrence, in the spirit of the temporal smoothing explored in [12], while adding concurrency and testability.

C. Algorithmic and Software Foundations

A greedy algorithm makes the locally best choice at every step and never revisits it [14]. Dynamic programming, introduced by Bellman [13], solves a problem by storing the solutions of overlapping sub-problems (memoisation) or by building a table of states in which each state depends on the previous one. Python's asyncio library provides cooperative multitasking based on an event loop, coroutines and tasks, and is well suited to workloads with many independent activities, such as polling many zones [15]. The producer–consumer pattern with a bounded queue decouples data generation from processing and applies back-pressure, and the object-oriented structure follows the principles of programming to an interface and the observer pattern [16].

III. System Architecture

A. Pipeline

The architecture is a pipeline in which every stage has one responsibility (Fig. 1). A *DemandSimulator* generates ride requests, expires old ones and changes the number of available drivers; one asyncio task per zone puts a *ZoneSnapshot* on a bounded queue every tick. A small pool of worker tasks takes snapshots and prices them. The *SurgeCalculator* computes the ratio and orchestrates the memo table (*DPCache*), the greedy rule (*GreedyTierStrategy*) and the smoothing recurrence. The *FareCalculator* converts the multiplier into a fare, and the *PriceBoard*, a lock-protected store of the latest quote and the history of each zone, serves the dashboard and the command-line interface.

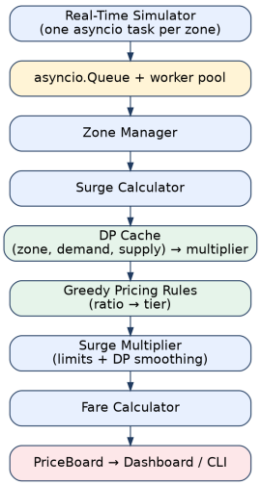


Fig. 1. Layered architecture of the surge pricing engine.

B. Concurrency

The engine follows the producer–consumer pattern. Five producer tasks, one per zone, run concurrently on one event loop. Each producer awaits *queue.put()*, so a slow consumer slows the producers instead of allowing unlimited memory growth. Three worker tasks wait on *queue.get()*. Pricing a snapshot is a purely synchronous section without any await; therefore two snapshots of the same zone are never interleaved and are priced in the order in which they were produced. This property matters because the smoothing state of a zone must be updated in time order. The producers are awaited with *asyncio.gather()*, *queue.join()* drains pending snapshots, and all tasks are then cancelled and awaited so that none is leaked.

C. Object-Oriented Design

Encapsulation is used throughout: *Zone* keeps its drivers and requests private, *DPCache* hides its ordered dictionary and *PriceBoard* hides its lock. *PricingEngine* is composed of *ZoneManager*, *DemandSimulator*, *SurgeCalculator*, *FareCalculator* and *PriceBoard*. *GreedyTierStrategy* extends the abstract *PricingStrategy*, so any other strategy can be plugged into the calculator. Frozen dataclasses carry immutable data (snapshots, quotes and configurations) across task and thread boundaries, and *PricingEngine.subscribe()* implements the observer pattern for the command-line interface. Tiers are data rather than if–else code, which makes the rule table configurable and easy to test.

D. Dashboard

The dashboard is a Streamlit script [17] with Plotly charts [18]. The asyncio loop runs in a daemon thread started by a cached runner object, and the page reads results only through the *PriceBoard*, whose methods are guarded by a *threading.Lock*. A fragment with auto-refresh redraws the key figures and charts without resetting the sidebar widgets. The sidebar controls the tariff, the maximum multiplier, the smoothing switch, the simulation speed and pause/resume (Fig. 2).

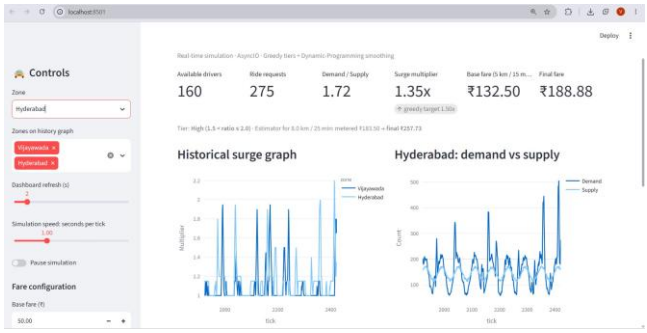


Fig. 2. Dashboard showing available drivers, ride requests, demand/supply ratio, surge multiplier, base fare, final fare and the historical surge graph.

IV. Algorithms

A. Ratio and Greedy Tier Selection

The ratio r is the number of pending requests divided by the number of available drivers. If there are no drivers and at least one request, r is treated as infinite; if there are no requests, $r = 0$. The greedy algorithm selects the pricing tier immediately from the current ratio, never looks at earlier ratios and never revises its decision. The tiers are upper-inclusive intervals (Table I), so that the boundary value $r = 2.0$ belongs to the 1.5× tier. The tier lookup scans a table with a constant number of entries and therefore takes O(1) time and space. For example, demand 80 with supply 100 gives $r = 0.8$ and 1.0×; demand 100 with supply 50 gives $r = 2.0$ and 1.5×; and demand 100 with supply 20 gives $r = 5.0$ and 2.5×.

TABLE I
Greedy Pricing Tiers

Ratio	Multiplier	Label
$r \le 1.0$	1.0×	Balanced
$1.0 < r \le 1.5$	1.2×	Busy
$1.5 < r \le 2.0$	1.5×	High
$2.0 < r \le 3.0$	2.0×	Very high
$r > 3.0$	2.5×	Extreme

B. Memoisation of Overlapping Sub-problems

Let G(zone, demand, supply) denote the greedy multiplier of a state. Identical states recur, especially in small zones and at night, so each solved sub-problem is stored in *DPCache* under the key (zone, demand, supply) and later requests are answered by a dictionary lookup. The cache is a bounded least-recently-used table with hit and miss counters, which limits memory use and makes the benefit of memoisation measurable. The cache stores the un-clamped greedy value so that the minimum and maximum limits can be changed at run time without invalidating it; the smoothing state depends on history and is deliberately not part of the key.

C. State Recurrence for Smoothing

The multiplier published for a zone should not differ much from the one published a moment ago. Let $G[t]$ be the greedy target after limits and $M[t-1]$ the previous multiplier of the same zone, kept in a table with one entry per zone. The new multiplier is

$$M[t] = \text{clamp}(M[t-1] + \text{clamp}(\alpha(G[t] - M[t-1]), -s, +s), m, M\text{max}) \quad (1)$$

with smoothing factor α (default 0.5; α = 1 disables smoothing), maximum step s per tick (default 0.5), limits m and Mmax (default 1.0 and 2.5) and cold start $M[0] = G[0]$. Because $M[t]$ depends only on $M[t-1]$ and $G[t]$, the table of states is filled from the first tick onwards and each entry is reused by the next, which is the bottom-up form of dynamic programming. The α term moves the price part of the way towards the target (exponential smoothing), and the s term guarantees that no single tick changes the price by more than 0.5×. Results are rounded to the nearest 0.05× so that riders see understandable numbers, and if the target is within one rounding unit of the previous value it is adopted directly so that the price converges exactly. Table II shows the effect when the ratio jumps from 0.8 to 5.0: the greedy rule asks for 2.5× immediately, whereas the published multiplier climbs 1.00, 1.50, 2.00, 2.25, 2.40, 2.45 and reaches 2.50 at tick 6.

TABLE II
Smoothing Example (α = 0.5, s = 0.5)

Tick	Greedy G[t]	Previous M[t−1]	Published M[t]
0	1.00	—	1.00
1	2.50	1.00	1.50
2	2.50	1.50	2.00
3	2.50	2.00	2.25
4	2.50	2.25	2.40
5	2.50	2.40	2.45
6	2.50	2.45	2.50

D. Fare Model

The fare is computed in three steps: the metered fare is the sum of the base fare, the distance charge and the time charge; it is multiplied by the surge multiplier and compared with the minimum fare; and a fixed booking fee is added:

$$F = \max((b + d \cdot k + \tau \cdot c) \cdot M, F\text{min}) + f \quad (2)$$

where b is the base fare, d and τ are the distance (km) and duration (min), k and c are the per-kilometre and per-minute rates, Fmin is the minimum fare and f is the booking fee. With the default tariff ($b = 50$, $k = 12$, $c = 1.5$, $f = 10$, Fmin = 80, in Indian rupees) a 5 km, 15 minute trip costs ₹142.50 at 1.0× and ₹275.00 at 2.0×.

E. Complexity

A cache hit costs O(1) average time; a miss costs O(1) for the greedy lookup plus O(1) for insertion; the smoothing step is O(1). With an LRU bound C and Z zones, memory is O(C) for the cache and O(Z) for the state table. Without the dynamic-programming layer the calculator would need no memory at all but could not remove sudden fluctuations; the trade-off is a small, bounded amount of memory.

V. Experimental Setup

Real ride-hailing data were not available, so demand and supply are generated by a simulator for five zones (Vijayawada, Hyderabad, Bangalore, Chennai and Mumbai), each with its own fleet size, base demand and demand pattern. Demand at tick t is $base \times (1 + 0.55 \sin 2\pi(t/period + phase))$, multiplied by an event factor between 1.8 and 2.6 during random events (probability 0.03 per tick, lasting 4 to 9 ticks), plus Gaussian noise. Requests stay pending for three ticks. Supply follows a smaller wave that lags demand by three ticks, so that demand outruns supply at peaks, which is exactly the situation in which surge pricing is needed. Each zone has its own random generator seeded with the global seed plus the CRC-32 of the zone name, so results are reproducible regardless of task scheduling order.

All reported experiments use seed 42 and the default configuration. The long run covers 300 ticks for each of the five zones (1500 snapshots). Stability is measured by the mean absolute change of the multiplier between consecutive ticks and by the number of single-tick jumps of at least 0.5×, comparing the greedy target alone with the smoothed published value. The correctness of the implementation is verified with a Pytest suite [19].

VI. Results

A. Greedy Target versus Smoothed Multiplier

Fig. 3 compares the raw greedy target with the published multiplier for Hyderabad over 120 ticks. The stepped greedy line changes abruptly, whereas the published line absorbs one-tick jumps and follows the same trend with a short delay. At the start of the sample run the greedy rule already asks for 2.0× at tick 2 (ratio 2.42), but the published multiplier is 1.75× and reaches 2.0× only at tick 7.

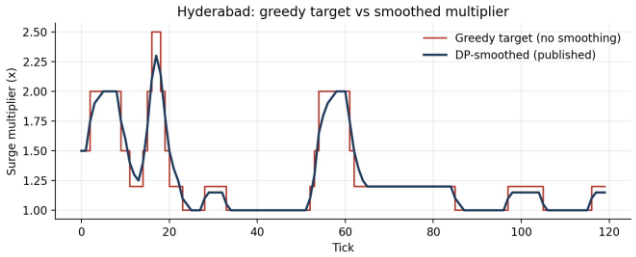


Fig. 3. Greedy target and DP-smoothed multiplier for Hyderabad (120 ticks).

B. Price Stability

Table III summarises the effect of smoothing. Over all zones the mean absolute tick-to-tick change fell from 0.0505 to 0.0395, a reduction of about 22%, and all 59 jumps of 0.5× or more produced by the greedy rule disappeared. The largest single-tick change fell from 0.80× to 0.45×. The mean multiplier is practically unchanged (within 0.02× in every zone, for example 1.24 against 1.22 in Mumbai), which shows that smoothing delays price changes without systematically raising or lowering prices. Fig. 4 shows the reduction in every zone.

TABLE III
Effect of smoothing (300 ticks per zone, seed 42)

Zone	Mean greedy	Mean publ.	Δ greedy	Δ smoothed	Jumps ≥0.5× (G/S)
Vijayawada	1.12	1.12	0.049	0.038	11 / 0
Hyderabad	1.22	1.21	0.057	0.048	18 / 0
Bangalore	1.19	1.18	0.050	0.037	10 / 0
Chennai	1.14	1.12	0.047	0.032	8 / 0
Mumbai	1.24	1.22	0.049	0.042	12 / 0
All zones	—	—	0.0505	0.0395	59 / 0

Note: Δ is the mean absolute change of the multiplier per tick; G/S = greedy/smoothed.

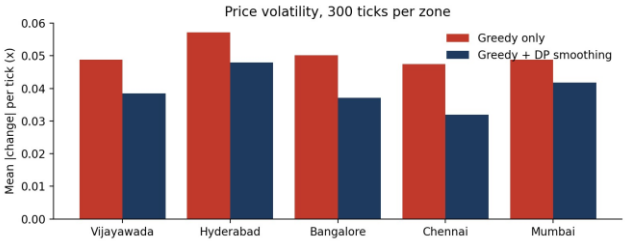


Fig. 4. Mean absolute multiplier change per tick, greedy only versus greedy with DP smoothing.

C. Concurrent Processing

All five zones are priced in the same run, each reacting to its own demand pattern (Fig. 5). Because the zone tasks run concurrently, one tick of five zones with a 0.1 s interval takes about 0.1 s instead of 0.5 s. In the automated timing test, five zones and five ticks completed in 0.53 s against 2.5 s for sequential processing, i.e. below 60% of the sequential time as required by the test.

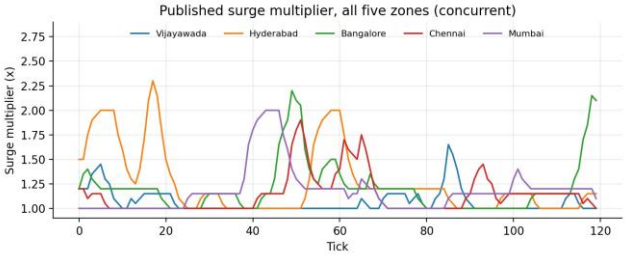


Fig. 5. Published surge multipliers of the five zones priced concurrently.

D. Dynamic-Programming Cache Statistics

Table IV reports the memo-table statistics. The hit rate grows with the length of the run because more states repeat, from 0.0% for 40 states to 13.9% for 3000 states. The numbers are modest because the exact (demand, supply) pair of a large zone repeats rarely. Since the greedy value is a pure function of the ratio and the tier table is small, memoisation saves little time here; its main value in this project is as a measurable demonstration of overlapping sub-problems, while the practical benefit comes from the smoothing recurrence.

TABLE IV
Memo-table statistics

Run	Hits	Misses	Hit rate
8 ticks × 5 zones (40 states)	0	40	0.0%
60 ticks × 5 zones (300)	6	294	2.0%
300 ticks × 5 zones (1500)	131	1369	8.7%
600 ticks × 5 zones (3000)	416	2584	13.9%

E. Verification

The suite consists of 45 Pytest test functions in three files which expand to 68 test cases through parametrisation, and all 68 passed in the development run. They cover the ratio and zero supply, every tier boundary (0.99, 1.0, 1.01, 1.5, 1.51, 2.0, 2.01, 3.0, 3.01 and infinity), the cache hit, miss, zone-specific key and eviction behaviour, smoothing up and down, convergence and zone independence, fare calculation with and without surge, the minimum fare, invalid input, simulator reproducibility, and the concurrency properties (all zones processed for every tick, per-zone ordering with several workers, determinism with the same seed, stop, and pause/resume).

VII. Discussion and Limitations

The results show that the greedy rule and the recurrence play different roles: the greedy rule gives an immediate, explainable decision, and the recurrence trades a short delay for stability. This delay is the main cost of smoothing: for a genuine, sustained surge the published price reaches the target a few ticks later than the greedy price, as in Table II, so the step limit and α must be chosen to balance responsiveness against affordability.

Several limitations should be stated. First, demand and supply are simulated, so the absolute numbers describe the simulator and not a real market. Second, the cache hit rate is modest when exact states rarely repeat; a production system could quantise demand and supply into buckets at the cost of precision. Third, a ratio-based tier table ignores rider elasticity and driver behaviour, which empirical work shows to be important [1], [5]. Fourth, the engine was not benchmarked against learning-based pricing methods such as those in [10]–[12]; the comparison is limited to the memoryless greedy baseline.

VIII. Conclusion and Future Work

The Ride-Hailing Surge Pricing Engine shows how classical algorithmic techniques and modern Python concurrency can be combined into a compact, understandable pricing system. The greedy tier rule converts the current demand/supply ratio into a multiplier immediately; the dynamic-programming layer stores solved states and smooths the price with a per-zone recurrence, which in the experiments removed every jump of 0.5× or more and reduced average price movement by about 22% without changing the average price level; and AsyncIO allowed five zones to be generated and priced at the same time, with 0.53 s against 2.5 s for sequential processing. The object-oriented design keeps components replaceable, and a suite of 68 test cases supports correctness.

Future work includes replacing the simulator with a real data stream and persistent storage, geospatial indexing (for example H3 cells) to price small areas and smooth across neighbouring cells, demand forecasting so that prices react to expected demand, quantised cache keys to raise the hit rate, rider-elasticity and driver-response models to choose multipliers that maximise completed trips, fairness rules such as caps during declared emergencies, and a REST interface with containerised, sharded deployment.

References

[1] M. Liu, T. I. Tunca, Y. Xu, and W. Zhu, "Market formation, pricing, and value generation in ride-hailing services," *Manuf. Serv. Oper. Manag.*, vol. 27, no. 5, 2025, doi: 10.1287/msom.2022.0502.

[2] J. C. Castillo, D. Knoepfle, and E. G. Weyl, "Matching and pricing in ride hailing: Wild goose chases and how to solve them," *Manage. Sci.*, 2025, doi: 10.1287/mnsc.2022.00096.

[3] J. C. Castillo, D. Knoepfle, and G. Weyl, "Surge pricing solves the wild goose chase," in *Proc. 2020 ACM Conf. Econ. Comput. (EC '17)*, 2017.

[4] J. Hall, C. Kendrick, and C. Nosko, "The effects of Uber's surge pricing: A case study," Uber Technologies and Univ. Chicago Booth School of Business, 2024.

[5] M. K. Chen and M. Sheldon, "Dynamic pricing in a labor market: Surge pricing and flexible work on the Uber platform," in *Proc. 2016 ACM Conf. Econ. Comput. (EC '16)*, 2025.

[6] P. Cohen, R. Hahn, J. Hall, S. Levitt, and R. Metcalfe, "Using big data to estimate consumer surplus: The case of Uber," National Bureau of Economic Research, Cambridge, MA, USA, NBER Working Paper 22627, 2024.

[7] S. Banerjee, C. Riquelme, and R. Johari, "Pricing in ride-share platforms: A queueing-theoretic approach," SSRN working paper, 2025.

[8] C. Yan, H. Zhu, N. Korolko, and D. Woodard, "Dynamic pricing and matching in ride-hailing platforms," *Naval Res. Logist.*, vol. 67, no. 8, pp. 705–724, 2025.

[9] D. Freund and G. van Ryzin, "Pricing fast and slow: Limitations of dynamic pricing mechanisms in ride-hailing," working paper, 2021.

[10] E. Yuan and P. Van Hentenryck, "Real-time pricing optimization for ride-hailing quality of service," in *Proc. 30th Int. Joint Conf. Artif. Intell. (IJCAI)*, 2021, pp. 3742–3748, doi: 10.24963/ijcai.2021/515.

[11] X. Huang, Y. Cheng, J. Jin, and A. Kou, "Research on dynamic subsidy based on deep reinforcement learning for non-stationary stochastic demand in ride-hailing," *Sustainability*, vol. 16, no. 15, Art. no. 6289, 2024, doi: 10.3390/su16156289.

[12] A. S. Gujral *et al.*, "FairRide: A cooperative-game approach to fair surge pricing in ridesharing," in *Proc. AAAI Spring Symp. Ser.* [Online]. Available: https://ojs.aaai.org/index.php/AAAI-SS/article/view/36863

[13] R. Bellman, *Dynamic Programming*. Princeton, NJ, USA: Princeton Univ. Press, 2025.

[14] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[15] Python Software Foundation, "asyncio — Asynchronous I/O," Python 3 documentation. [Online]. Available: https://docs.python.org/3/library/asyncio.html

[16] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1994.

[17] Streamlit Inc., "Streamlit documentation." [Online]. Available: https://docs.streamlit.io

[18] Plotly Technologies, "Plotly Python graphing library." [Online]. Available: https://plotly.com/python/

[19] Pytest developers, "pytest documentation." [Online]. Available: https://docs.pytest.org